Phase 1 Integration Guide

Overview

This guide explains how to integrate Phase 1 performance components with the existing BDI kernel and Master Memory Manager.

Architecture Integration

Master Memory Manager (MMM) Integration

Phase 1 components integrate with MMM at multiple levels:

1. Shared Arena Integration

```
// In shared_arena.c
#include "../../master_memory_manager/master_memory_manager.h"

shared_arena_t* shared_arena_create_with_mmm(size_t size) {
    // Allocate backing memory from MMM
    mmm_config_t mmm_config = {
        .enable_x86_core = true,
        .memory_pool_size = size,
        .page_size = 4096
    };

    mmm_status_t status = mmm_initialize(&mmm_config);
    if (status != MMM_SUCCESS) {
        return NULL;
    }

    // Create arena on top of MMM-managed memory
    shared_arena_t* arena = shared_arena_create(size);
    arena->mmm_handle = (void*)&mmm_config;  // Store MMM handle

    return arena;
}
```

2. Capability Integration

```
// Integrate with MMM's memory protection
bool mmm_validate_capability(const capability_t* cap, void* ptr, size_t size) {
    // Use MMM's memory protection to validate access
    return capability_validate(cap, ptr, size, CAP_PERM_READ | CAP_PERM_WRITE);
}
```

X86 Core Integration

1. Memory Fences

Phase 1 uses x86-64 memory fence instructions:

```
// In ring_common.h
static inline void memory_fence_full(void) {
    __asm__ __volatile__("mfence" ::: "memory");
}

static inline void memory_fence_load(void) {
    __asm__ __volatile__("lfence" ::: "memory");
}

static inline void memory_fence_store(void) {
    __asm__ __volatile__("sfence" ::: "memory");
}
```

2. Atomic Operations

```
// Use x86-64 LOCK prefix for atomic operations
static inline bool atomic_cas(atomic_size_t* ptr, size_t expected, size_t desired) {
    return __atomic_compare_exchange_n(ptr, &expected, desired,
                                       false,
                                       __ATOMIC_ACQ_REL,
                                       __ATOMIC_ACQUIRE);
}
```

3. Fiber Context Switching

```
// In fiber.c - uses x86-64 registers
void fiber_switch(fiber_t* from, fiber_t* to) {
    // Save/restore callee-saved registers: RSP, RBP, RBX, R12-R15
    // See fiber.c for full implementation
}
```

Orchestrator Integration

1. Graph Call Routing

```
// In orchestrator integration
void orchestrator_register_graph_call_handler(graph_call_type_t type,
                                              graph_call_handler_t handler) {
    // Register handler for specific graph call type
    g_graph_call_handlers[type] = handler;
}

void orchestrator_route_graph_call(graph_call_request_t* request) {
    // Route to appropriate handler
    graph_call_handler_t handler = g_graph_call_handlers[request->type];
    if (handler) {
        handler(request);
    }
}
```

2. Fiber Scheduling Integration

```
// Integrate fiber scheduler with orchestrator
void orchestrator_schedule_fiber(fiber_t* fiber) {
    uint32_t core_id = get_current_core_id();
    fiber_scheduler_t* scheduler = get_scheduler_for_core(core_id);

    // Add fiber to scheduler's ready queue
    enqueue_fiber(scheduler, fiber);
}
```

Build Integration

CMake Integration

Add Phase 1 to the main BDI kernel build:

```
# In moduler_kernel/CMakeLists.txt
add_subdirectory(performance/phase1)

# Link Phase 1 libraries
target_link_libraries(bdi_kernel
    phase1_rings
    phase1_fibers
    phase1_arena
    phase1_ipc
    phase1_integration
)
```

Header Integration

```
// In moduler_kernel/integration/bdi_modular_main.c
#include "performance/phase1/integration/phase1_init.h"

int main(void) {
    // Initialize Phase 1
    phase1_config_t config = phase1_get_default_config();
    phase1_init(&config);

    // Initialize other BDI components
    // ...

    // Run kernel
    // ...

    // Shutdown
    phase1_shutdown();
    return 0;
}
```

Usage Examples

Example 1: Zero-Copy IPC Between Modules

```
// Module A (sender)
void module_a_send_data(void) {
    // Allocate buffer from shared arena
    void* buffer = shared_arena_alloc(g_arena, 4096);

    // Write data
    memcpy(buffer, data, data_size);

    // Create capability
    capability_t cap = capability_create(buffer, 4096,
                                         CAP_PERM_READ,
                                         CAP_TRUST_USER, 0, 0);

    // Create descriptor
    memory_descriptor_t desc = descriptor_create(buffer, 4096, cap, 0, 0);

    // Send via graph call (no syscall!)
    graph_call_request_t request = {
        .type = GRAPH_CALL_IPC_SEND,
        .params.ipc_send = {
            .descriptor = desc,
            .target_core = 1
        }
    };

    graph_call_submit(g_port, &request);
    graph_call_wait(g_port, &request, 0);
}

// Module B (receiver)
void module_b_receive_data(void) {
    // Receive descriptor
    graph_call_request_t request = {
        .type = GRAPH_CALL_IPC_RECV,
        .params.ipc_recv = {
            .required_perms = CAP_PERM_READ
        }
    };

    graph_call_submit(g_port, &request);
    graph_call_wait(g_port, &request, 0);

    // Map descriptor (no copy!)
    void* ptr;
    size_t len;
    zero_copy_map(&request.result.descriptor, CAP_PERM_READ, &ptr, &len);

    // Access data directly
    process_data(ptr, len);

    // Unmap
    zero_copy_unmap(&request.result.descriptor);
}
```

Example 2: Fiber-Based Concurrent Processing

```
void process_request(void* arg) {
    request_t* req = (request_t*)arg;

    // Process request
    result_t result = handle_request(req);

    // Yield to allow other fibers to run
    fiber_scheduler_yield(g_scheduler);

    // Send response
    send_response(&result);
}

void handle_requests(void) {
    // Spawn fibers for each request
    for (int i = 0; i < num_requests; i++) {
        fiber_scheduler_spawn(g_scheduler,
                              process_request,
                              &requests[i],
                              0,
                              FIBER_PRIORITY_NORMAL);
    }

    // Run scheduler (fibers execute cooperatively)
    fiber_scheduler_run(g_scheduler);
}
```

Example 3: Lock-Free Cross-Core Communication

```
// Core 0 (producer)
void core0_send_message(void) {
    message_t* msg = create_message();

    // Enqueue in MPSC ring (lock-free!)
    while (mpsc_ring_enqueue(g_cross_core_ring, msg) != RING_SUCCESS) {
        // Retry if full
        __asm__ __volatile__("pause");
    }
}

// Core 1 (consumer)
void core1_receive_messages(void) {
    while (1) {
        message_t* msg = NULL;
        if (mpsc_ring_dequeue(g_cross_core_ring, (void**)&msg) == RING_SUCCESS) {
            process_message(msg);
        }
    }
}
```

Performance Monitoring

Collecting Statistics

```
// Ring buffer statistics
ring_stats_t ring_stats;
spsc_ring_get_stats(ring, &ring_stats);
printf("Ring: %lu enqueues, %lu dequeues, %lu peak size\n",
       ring_stats.total_enqueues,
       ring_stats.total_dequeues,
       ring_stats.peak_size);

// Fiber scheduler statistics
uint64_t switches, yields, fibers;
fiber_scheduler_get_stats(scheduler, &switches, &yields, &fibers);
printf("Scheduler: %lu switches, %lu yields, %lu fibers\n",
       switches, yields, fibers);

// Arena statistics
arena_stats_t arena_stats;
shared_arena_get_stats(arena, &arena_stats);
printf("Arena: %lu allocs, %lu frees, %lu peak allocated\n",
       arena_stats.total_allocations,
       arena_stats.total_frees,
       arena_stats.peak_allocated);
```

Performance Counters

```
// Enable performance counters in config
phase1_config_t config = phase1_get_default_config();
config.enable_performance_counters = true;
phase1_init(&config);

// Counters are automatically collected and can be exported
// to MMM's monitoring system
```

Testing Integration

Unit Tests

```
cd moduler_kernel/performance/phase1/build
ctest --verbose
```

Integration Tests

```
// Test Phase 1 with MMM
void test_phase1_mmm_integration(void) {
    // Initialize MMM
    mmm_config_t mmm_config = {
        .enable_x86_core = true,
        .memory_pool_size = 64 * 1024 * 1024
    };
    mmm_initialize(&mmm_config);

    // Initialize Phase 1
    phase1_config_t phase1_config = phase1_get_default_config();
    phase1_init(&phase1_config);

    // Test operations
    // ...

    // Cleanup
    phase1_shutdown();
    mmm_shutdown();
}
```

Troubleshooting

Common Issues

1. **Ring buffer full errors**
   - Increase ring capacity in config
   - Check consumer is processing fast enough

2. **Fiber stack overflow**
   - Increase fiber stack size
   - Check for deep recursion

3. **Arena fragmentation**
   - Increase arena size
   - Use size classes more effectively

4. **CAS contention in MPSC**
   - Reduce number of producers
   - Use batching to reduce atomic operations

Debug Mode

```
// Enable debug mode for verbose logging
#define PHASE1_DEBUG 1

// Compile with debug symbols
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

Migration Path

From Traditional Syscalls to Graph Calls

1. Identify syscall-heavy code paths
2. Replace with graph call equivalents
3. Measure performance improvement
4. Iterate on hot paths

From Threads to Fibers

1. Identify context-switch heavy code
2. Convert thread-based to fiber-based
3. Add explicit yield points
4. Test cooperative scheduling

From Locks to Lock-Free

1. Identify lock-contended code
2. Replace with SPSC/MPSC rings
3. Ensure proper memory ordering
4. Stress test for correctness

Next Steps

After integrating Phase 1:

1. **Measure baseline performance** against Linux
2. **Identify bottlenecks** using profiling tools
3. **Optimize hot paths** with Phase 1 primitives
4. **Plan Phase 2** optimizations based on results

Support

For issues or questions:
- Check documentation in `docs/`
- Review test cases in `tests/`
- Examine benchmarks in `bench/`
- Consult architecture guide in `docs/ARCHITECTURE.md`